

Java Backend Interview Series

Sub-Chapter 1.13: JVM Internals

Class Loading | Memory Areas | GC Algorithms | JIT | Bytecode | Tuning

52 Q&A | Advanced Questions | Exercises | Cheat Sheet

1. JVM Architecture & Runtime Data Areas

The JVM is a stack-based virtual machine with clearly partitioned runtime memory areas. Understanding these areas is essential for diagnosing OOM errors, tuning performance, and debugging memory leaks.

Memory Area	Per-Thread?	Contents	OOM Type
Heap	No (shared)	Objects, arrays, class instances	java.lang.OutOfMemoryError : Java heap space
Method Area	No (shared)	Class metadata, static fields, constants (Metaspace in Java 8+)	OOM: Metaspace
JVM Stack	Yes	Stack frames: local variables, operand stack, frame data	StackOverflowError
PC Register	Yes	Address of current executing bytecode instruction	Never OOM
Native Stack	Yes	Native method call frames (JNI)	StackOverflowError (native)
Code Cache	No (shared)	JIT-compiled native code	OOM: Code Cache (CodeCache is full)
Direct Memory	No (shared)	Off-heap NIO ByteBuffer allocations	OOM: Direct buffer memory

```
// Heap monitoring via Runtime
Runtime rt = Runtime.getRuntime();
long maxHeap = rt.maxMemory(); // -Xmx
long totalHeap = rt.totalMemory(); // current committed heap
long freeHeap = rt.freeMemory(); // free in committed heap
long usedHeap = totalHeap - freeHeap;

// Metaspace tuning flags
// -XX:MetaspaceSize=128m -- initial metaspace size
// -XX:MaxMetaspaceSize=256m -- cap metaspace (default: unlimited!)
// -XX:+UseCompressedClassPointers -- compress klass pointers (64-bit JVMs)

// Stack depth tuning
// -Xss512k -- per-thread stack size (default 512k-1m)
// Smaller -Xss = more threads but shallower recursion

// Code Cache
// -XX:InitialCodeCacheSize=32m
// -XX:ReservedCodeCacheSize=256m
// -XX:+PrintCodeCache -- print code cache usage on exit
```

Metaspace vs PermGen

Before Java 8: PermGen (Permanent Generation) was part of the heap with a fixed upper bound (-XX:MaxPermSize). OOM: PermGen space was a common cause of application restarts. Java 8+: PermGen replaced by Metaspace -- allocated in native memory, grows dynamically. No MaxMetaspaceSize cap by default -- class metadata leaks can consume all OS memory. Always set -XX:MaxMetaspaceSize in production.

2. Class Loading: Bootstrap, Extension, Application & Custom Loaders

The class loading subsystem is responsible for loading, linking, and initialising classes. The **delegation model** (parent-first) ensures core JDK classes are always loaded by the bootstrap loader, preventing spoofing. Custom classloaders enable hot-reload, OSGi, and isolation (application servers, plugins).

```
// ClassLoader hierarchy
// Bootstrap ClassLoader -- loads rt.jar / java.* (written in C, null parent)
// |-- Platform ClassLoader (Java 9+, formerly Ext CL) -- java.ext.dirs
// |-- App ClassLoader -- -classpath / CLASSPATH
```

```
// |-- Custom ClassLoaders
// Delegation model: child asks parent first
class DatabaseDriverLoader extends ClassLoader {
@OVERRIDE
protected Class<?> findClass(String name) throws ClassNotFoundException {
if (name.startsWith("com.mydb.")) {
byte[] bytes = loadClassBytes(name); // read from DB or JAR
return defineClass(name, bytes, 0, bytes.length);
}
return super.findClass(name); // delegate to parent
}
}

// Class loading phases
// 1. Loading -- read .class bytes, create Class object in Metaspace
// 2. Verification -- verify bytecode safety (no illegal jumps, type safety)
// 3. Preparation -- allocate static fields with default values (0/null/false)
// 4. Resolution -- resolve symbolic references to direct references
// 5. Initialisation -- execute <clinit> (static initialisers, in order)

// Inspect class loading
ClassLoader cl = MyClass.class.getClassLoader();
System.out.println(cl); // sun.misc.Launcher$AppClassLoader
System.out.println(cl.getParent()); // Platform/Extension CL
System.out.println(cl.getParent().getParent()); // null (Bootstrap)

// -verbose:class -- log every class load/unload
// -XX:+TraceClassLoading
// -XX:+TraceClassUnloading
```

Phase	Key Action	Flag to Monitor
Loading	Read .class bytes; create Class object	-verbose:class
Verification	Bytecode safety check (type, jump targets)	-Xverify:all / :none
Preparation	Static fields get default values	N/A
Resolution	Symbolic refs -> direct refs (lazy/eager)	N/A
Initialisation	Run static blocks; execute	-XX:+TraceClassInitialization

3. Heap Structure & Generational Garbage Collection

The JVM heap is divided into generations based on the **generational hypothesis**: most objects die young. This allows GC to focus effort on short-lived objects (Young Generation) and rarely scan long-lived objects (Old Generation).

```
// Heap layout (Parallel / Serial GC -- traditional generational)
//
// Young Generation (Eden + Survivor S0 + Survivor S1)
// |-- Eden: new objects allocated here (TLAB)
// |-- S0/S1: objects that survived one or more Minor GCs
//
// Old Generation (Tenured):
// |-- Objects promoted after surviving N GCs (default: 15)
// |-- Large objects allocated directly (Large Object Threshold)
//
// Minor GC: collects Young Gen only (stop-the-world, fast)
// Major/Full GC: collects Old Gen (and sometimes Young) -- longer pause

// Key JVM flags for heap sizing
// -Xms512m -- initial heap size
// -Xmx2g -- max heap size
```

```
// -Xmn512m -- young generation size
// -XX:SurvivorRatio=8 -- Eden:Survivor ratio (8 = Eden is 8/10 of Young)
// -XX:MaxTenuringThreshold=15 -- GC cycles before promotion to Old Gen
// Object allocation path:
// 1. TLAB (Thread-Local Allocation Buffer) in Eden -- fastest (bump pointer)
// 2. Eden (with lock, if TLAB full)
// 3. Old Gen (if object > -XX:PretenureSizeThreshold or Eden full)
```

GC Event	Scope	Pause Type	Trigger
Minor GC	Young Gen only	Stop-the-world (short)	Eden full
Major GC	Old Gen	Stop-the-world (long)	Old Gen fills up
Full GC	Entire heap + Metaspace	Stop-the-world (longest)	System.gc(), OOM risk
Concurrent	Runs with app threads	Mostly concurrent (G1/ZGC/Shenandoah)	Scheduled by GC

4. GC Algorithms: Serial, Parallel, G1, ZGC, Shenandoah

Java offers multiple GC algorithms for different workload characteristics. Choosing the right GC is one of the most impactful performance decisions.

GC	Flag	Pause Model	Heap Size	Best For
Serial GC	-XX:+UseSerialGC	STW all phases	Small	Single-core, embedded
Parallel GC	-XX:+UseParallelGC	STW, parallel threads	Medium	Batch/throughput focus
G1 GC	-XX:+UseG1GC	Mostly STW, concurrent mark	>=4GB	General purpose, default Java 9+
ZGC	-XX:+UseZGC	Sub-millisecond (concurrent)	Any	Ultra-low latency, Java 15+ prod
Shenandoah	-XX:+UseShenandoahGC	Near-zero pause (concurrent)	Any	Low latency, Red Hat JDK

```
// G1 GC -- key concepts
// - Heap divided into equal-sized regions (1-32 MB each)
// - Young regions: Eden + Survivor (dynamically resized)
// - Old regions: tenured objects
// - Humongous regions: objects > 50% of region size
// - Concurrent marking determines liveness of old regions
// - Mixed collections: young + some old regions (most-garbage-first)
// G1 tuning
// -XX:MaxGCPauseMillis=200 -- pause time goal (not hard limit)
// -XX:G1HeapRegionSize=16m -- region size (auto if not set)
// -XX:G1NewSizePercent=5 -- min young gen %
// -XX:G1MaxNewSizePercent=60 -- max young gen %
// -XX:G1MixedGCLiveThresholdPercent=85 -- only mix regions below this liveness
// ZGC -- key properties (Java 15+ production)
// - All GC work except STW pauses is concurrent with application
// - STW pauses: 3 (initial mark, remark, relocation start) -- each < 1ms
// - Load barriers: read barriers rewrite pointers on-the-fly (colored pointers)
// - Scalable to terabytes of heap
// -XX:+UseZGC -XX:ZAllocationSpikeTolerance=5
// Epsilon GC (Java 11) -- no-op GC for benchmarking
// -XX:+UseEpsilonGC -- allocates but never collects; OOM when heap full
```

GC Tuning Principle

Start with defaults (G1 on Java 9+). Only tune after profiling with actual workload. Key metrics: pause time (GC logs), allocation rate (GC logs), throughput (reduced GC overhead). Enable GC logging:
`-Xlog:gc*:file=gc.log,time,uptime,level,tags:filecount=5,filesize=20m`

5. JIT Compilation: C1, C2, Tiered Compilation & Optimisations

The JVM interprets bytecode initially, then uses **Just-In-Time (JIT) compilation** to compile hot methods to native code. HotSpot uses **tiered compilation** (C1 for quick profiling, C2 for aggressive optimisation) for optimal startup and peak performance.

```
// Tiered Compilation levels (Java 7+, enabled by default)
// Level 0: Interpreter
// Level 1: C1 -- simple JIT, no profiling (trivial methods)
// Level 2: C1 -- with light profiling (invocation + backedge counters)
// Level 3: C1 -- with full profiling (type profiles, branch profiles)
// Level 4: C2 -- optimising JIT (uses profile data for aggressive opts)
// Key JIT optimisations
// 1. Method inlining -- replace call site with callee body
// (most impactful: eliminates call overhead, enables further opts)
// -XX:MaxInlineSize=35 -- bytecodes limit for inlining
// -XX:FreqInlineSize=325 -- for frequently-called methods
//
// 2. Escape analysis -- if object doesn't escape method: stack-allocate
// -XX:+DoEscapeAnalysis (default on)
//
// 3. Lock elision -- remove locks on non-escaping objects
//
// 4. Loop unrolling -- replicate loop body N times, reduce iterations
//
// 5. Dead code elimination -- remove unreachable/unused code
//
// 6. Devirtualisation -- if only one impl seen, inline virtual call
// (monomorphic: 1 target, bimorphic: 2, megamorphic: 3+ -- no inline)
// JIT diagnostic flags
// -XX:+PrintCompilation -- print each compiled method
// -XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining -- print inlining decisions
// -XX:CompileThreshold=10000 -- invocations before C2 compile (default)
// -Xcomp -- compile everything on first call (no warmup)
// -Xint -- interpret only (disable JIT)
```

Warmup & JIT Traps

JVM performance benchmarks must account for warmup (JIT compilation). A method needs ~10,000 invocations before C2 compilation by default. Megamorphic call sites (3+ different receiver types) prevent devirtualisation and inlining -- a common performance trap in heavily polymorphic code. Use JMH (Java Microbenchmark Harness) for accurate Java benchmarks.

6. Bytecode, Class File Structure & javap

Java source code compiles to platform-neutral **bytecode** (.class files). Understanding bytecode is essential for debugging, performance analysis, and framework development (AspectJ, Spring AOP, Lombok, etc.).

```
// Java source
public int add(int a, int b) { return a + b; }
// Compiled bytecode (javap -c MyClass)
```

```
// public int add(int, int);
// Code:
// 0: iload_1 -- push local variable slot 1 (a) onto operand stack
// 1: iload_2 -- push local variable slot 2 (b)
// 2: iadd -- pop 2 ints, push their sum
// 3: ireturn -- pop and return int from top of stack
// Class file structure (magic: 0xCAFEBAFE)
// Magic (4 bytes: 0xCAFEBAFE) | Minor version | Major version
// Constant Pool (strings, class refs, method refs, field refs)
// Access flags (public, final, interface, abstract, synthetic...)
// This class | Super class | Interfaces
// Fields | Methods | Attributes (SourceFile, LineNumberTable, ...)
// Major version numbers
// Java 8 = 52, Java 11 = 55, Java 17 = 61, Java 21 = 65
// javap commands
// javap -c MyClass -- disassemble bytecodes
// javap -v MyClass -- verbose: constant pool, stack sizes
// javap -p MyClass -- show private members
// javap -l MyClass -- line number + local variable tables
```

```
// Common bytecode instructions
// iload_N, lload_N, fload_N, dload_N, aload_N -- push local var
// istore_N, lstore_N ... -- pop to local var
// iadd, isub, imul, idiv, irem -- int arithmetic
// invokevirtual -- virtual method dispatch (polymorphic)
// invokespecial -- constructors, private, super
// invokestatic -- static methods
// invokeinterface -- interface method dispatch
// invokedynamic -- dynamic dispatch (lambdas, MethodHandles)
// new -- allocate object (does NOT call constructor)
// dup -- duplicate top of stack (needed after new)
// invokespecial <init> -- call constructor
// checkcast -- type cast check
// instanceof -- type check
// getfield/putfield -- field access
// getstatic/putstatic -- static field access
// tableswitch / lookupswitch -- switch statement implementations
// goto -- unconditional jump
// if_icmplt, if_icmpeq, ... -- conditional jumps
```

7. GC Tuning, Memory Leaks & Diagnostic Tools

Diagnosing memory issues requires a combination of GC log analysis, heap dumps, and profiling tools. Common problems: memory leaks, excessive GC pauses, metaspace exhaustion, and direct memory leaks.

```
// Enable comprehensive GC logging (Java 9+)
// -Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=20m
// Heap dump on OOM
// -XX:+HeapDumpOnOutOfMemoryError
// -XX:HeapDumpPath=/var/log/heapdump.hprof
// On-demand heap dump
jcmd <pid> GC.heap_dump /path/to/dump.hprof
jmap -dump:format=b,file=heap.hprof <pid>
// Heap dump analysis
// Eclipse MAT (Memory Analyser Tool) -- find leak suspects, dominator tree
// jhat (deprecated) -- built-in heap analyser
```

```
// VisualVM -- GUI profiler + heap analyser
// GC overhead monitoring
// jstat -gcutil <pid> 1000 10 -- GC stats every 1s, 10 samples
// Output: S0 S1 E O M YGC YGCT FGC FGCT GCT
// S0/S1: Survivor usage %, E: Eden %, O: Old %, M: Metaspace %
// Memory leak patterns
// 1. Static collections growing unboundedly
// 2. ThreadLocal not removed (thread pool)
// 3. Listeners not deregistered (observer pattern)
// 4. Cache with no eviction (use WeakReference or bounded cache)
// 5. ClassLoader leaks (dynamic loading without unloading)
// Weak/Soft/Phantom references
WeakReference<Cache> weak = new WeakReference<>(cache);
// GC: eligible when only weakly reachable (cleared at next GC)
SoftReference<BigImage> soft = new SoftReference<>(image);
// GC: cleared only when memory is low (good for caches)
PhantomReference<Object> phantom = new PhantomReference<>(obj, queue);
// GC: enqueued after object finalized; cannot call get() (returns null)
```

Reference Type	Cleared By GC?	get() After Clear	Use Case
Strong	No (prevents GC)	Yes	Normal references
Soft	When memory low	null	Memory-sensitive caches
Weak	At any GC	null	Canonicalization, metadata
Phantom	After finalization	Always null	Pre-mortem cleanup, resource tracking

8. Java Flight Recorder, JFR Events & Performance Profiling

Java Flight Recorder (JFR) is a low-overhead production profiler built into the JDK. It records JVM events (GC, thread, I/O, lock, method profiling) with typically less than 1% overhead. Combined with **JDK Mission Control (JMC)**, it provides deep performance analysis.

```
// JFR -- start from command line
java -XX:StartFlightRecording=duration=60s,filename=recording.jfr MyApp
// JFR -- via jcmd (on running process)
jcmd <pid> JFR.start name=profiling duration=120s filename=rec.jfr
jcmd <pid> JFR.stop name=profiling
jcmd <pid> JFR.dump name=profiling filename=dump.jfr
// JFR -- programmatic API (Java 14+)
Recording recording = new Recording();
recording.enable("jdk.GarbageCollection").withThreshold(Duration.ofMillis(10));
recording.enable("jdk.CPULoad").withPeriod(Duration.ofSeconds(1));
recording.start();
// ... run workload ...
recording.stop();
recording.dump(Path.of("recording.jfr"));
// JFR -- custom events
@Name("com.myapp.RequestEvent")
@Label("HTTP Request")
@Category("MyApp")
class RequestEvent extends Event {
    @Label("Request URL") String url;
    @Label("Duration ms") long durationMs;
}
RequestEvent event = new RequestEvent();
event.url = request.getUrl();
```

```

event.begin();
// ... process request ...
event.end();
event.commit(); // records only if recording is active
// Key JFR events to analyse
// jdk.GarbageCollection -- GC pause duration, cause
// jdk.CPULoad -- JVM + system CPU usage
// jdk.JavaMonitorWait -- lock contention
// jdk.ThreadSleep -- thread sleep events
// jdk.SocketRead/Write -- I/O profiling
// jdk.ClassLoad -- class loading
// jdk.MethodProfiling -- hot method sampling

```

Tool	Purpose	Overhead	Key Command / Usage
jstat	GC stats (live)	Negligible	jstat -gcutil 1s
jstack	Thread dump	Brief STW	jstack
jmap	Heap dump / class histogram	Pauses JVM	jmap -histo
jcmd	Swiss army knife	Varies	jcmd help
JFR	Production profiler	<1%	-XX:StartFlightRecording
JMC	JFR analysis GUI	N/A	Open .jfr file
VisualVM	Profiling + heap analysis	2-5%	visualvm --jdkhome \$JAVA_HOME
async-profiler	Low-overhead CPU/alloc profiler	<1%	./profiler.sh -d 30 -f out.html

Interview Q&A; - Standard Questions

Q1. What are the main runtime data areas in the JVM?

The JVM runtime data areas are: (1) **Heap** -- shared, stores all objects and arrays; (2) **Method Area / Metaspace** -- shared, stores class metadata, static fields, constant pool; (3) **JVM Stack** -- per-thread, stores stack frames with local variables and operand stack; (4) **PC Register** -- per-thread, address of current bytecode instruction; (5) **Native Method Stack** -- per-thread, for native (JNI) method calls; (6) **Code Cache** -- shared, JIT-compiled native code.

Q2. What is the difference between the heap and the stack in Java?

The **heap** is a single shared memory area where all objects and arrays are allocated. Objects live until GC reclaims them (no fixed lifetime). Accessed by multiple threads. The **stack** is per-thread. Each method call creates a stack frame containing: local variables, operand stack, and return address. Stack frame is destroyed when the method returns. Primitive local variables and object references live on the stack; the objects they point to live on the heap.

Q3. What is the difference between PermGen and Metaspace?

PermGen (pre-Java 8): fixed-size region within the heap holding class metadata, interned strings, and static variables. Prone to 'OOM: PermGen space' errors. Metaspace (Java 8+): class metadata moved to native memory outside the heap. Grows dynamically -- no default upper bound (set -XX:MaxMetaspaceSize). Interned strings moved to the heap. Benefit: eliminates most OOM: PermGen errors; risk: unbounded native memory consumption.

Q4. Explain the class loading process phases.

Five phases: (1) **Loading** -- read .class bytes, create Class object in Metaspace. (2) **Verification** -- validate bytecode correctness (type safety, valid jumps). (3) **Preparation** -- allocate static fields, set to default values (0/null/false). (4) **Resolution** -- resolve symbolic references (class/field/method names) to direct references. (5) **Initialisation** -- execute static initialiser blocks (clinit) in order. Loading+Verification+Preparation+Resolution = Linking phase.

Q5. What is the parent delegation model of class loading?

When a ClassLoader is asked to load a class, it first delegates to its parent. Only if the parent cannot find it does the child attempt to load. Hierarchy: Bootstrap CL (rt.jar/java.*) -> Platform CL (java.ext) -> App CL (classpath). Benefits: (1) core JDK classes always loaded by Bootstrap -- prevents spoofing java.lang.String; (2) avoids loading the same class twice by different loaders (which would be incompatible types). Override: child-first loaders (e.g., OSGi, Tomcat webapp classloader) break delegation for isolation.

Q6. What is the generational hypothesis and why does it matter for GC?

The **weak generational hypothesis**: most objects die young (live for a very short time). Evidence: 80-98% of objects become unreachable within milliseconds in typical Java applications. Implication for GC: focus collection effort on young objects (Eden/Survivor). Minor GC: collects only the Young Generation -- fast because most objects are dead (few survivors). Old Generation GC is expensive and happens rarely. This hypothesis enables generational GC algorithms to achieve high throughput.

Q7. What is a Minor GC and what triggers it?

Minor GC (Young GC) collects the Young Generation (Eden + Survivor spaces). Trigger: Eden space is full when a thread tries to allocate. Algorithm: mark live objects, copy survivors to the other Survivor space (copying collector -- no fragmentation). Objects surviving a threshold number of GCs are promoted to Old Gen. Stop-the-world but typically short (milliseconds) because most objects in Eden are dead.

Q8. What is a Full GC and what are its causes?

Full GC collects the entire heap (Young + Old + Metaspace). Stop-the-world, longest pause. Causes: (1) Old Generation full; (2) Metaspace full; (3) System.gc() called; (4) explicit RMI GC; (5) CMS/G1 concurrent mode failure (concurrent marking can't keep up). Prevention: size heap correctly, avoid System.gc(), tune GC for your workload, use G1/ZGC for concurrent collection of old gen.

Q9. What is G1 GC and how does it differ from Parallel GC?

G1 (Garbage First, default since Java 9): divides heap into equal-sized regions (1-32 MB). Regions are dynamically assigned as Eden, Survivor, Old, or Humongous. Concurrent marking runs alongside the application to identify garbage regions. Mixed GC: collects young regions + old regions with most garbage (highest ROI first). Pause time goal: -XX:MaxGCPauseMillis=200 (best-effort). Parallel GC: stop-the-world but highly parallel; maximises throughput; no pause time goal.

Q10. What is tiered compilation in HotSpot?

HotSpot uses 5 tiers: Level 0 (interpreter), Level 1-3 (C1 JIT with increasing profiling), Level 4 (C2 optimising JIT). Initially methods run interpreted (tier 0). After ~2000 invocations: C1 (tier 3) with profiling. After ~15000 invocations and profile data collected: C2 (tier 4) with aggressive optimisations. Tiered compilation balances fast startup (C1 quickly compiled) with peak performance (C2 optimised).

Q11. What is method inlining and why is it the most important JIT optimisation?

Method inlining replaces a call site with the body of the called method. Benefits: (1) eliminates call overhead (frame creation, argument passing); (2) makes more code visible to the optimiser -- enables further opts like dead code elimination, constant folding, escape analysis; (3) enables devirtualisation in combination with type profiles. The JIT inlines methods up to -XX:MaxInlineSize=35 bytecodes and very frequently called methods up to -XX:FreqInlineSize=325.

Q12. What is escape analysis and what optimisations does it enable?

Escape analysis determines whether an object's lifetime is bounded to a method or thread. Types: (1) **No escape** -- object does not escape the allocating method: stack allocation or scalar replacement (explode into local variables). (2) **Thread escape only** -- only accessed by one thread: lock elision. (3) **Global escape** -- accessible by other threads: heap allocation. Enabled by default: -XX:+DoEscapeAnalysis. Reduces heap allocation, GC pressure, and lock overhead.

Q13. What causes a StackOverflowError?

StackOverflowError occurs when the JVM Stack exceeds its maximum size. Cause: infinite or excessively deep recursion. Each method call creates a new stack frame -- recursive calls accumulate frames until the stack limit is reached. The default stack size is 512KB-1MB (-Xss flag). Fix: (1) add a base case to the recursion; (2) convert to iterative with explicit stack; (3) increase -Xss (but risks more memory per thread). Note: StackOverflowError is an Error, not Exception -- do not catch and swallow it.

Q14. What is a WeakReference and when would you use it?

A WeakReference allows GC to collect the referent object even if the WeakReference exists. The reference is cleared at any GC. get() returns null after collection. Use cases: (1) canonical maps -- WeakHashMap entries are GC'd when key has no strong references; (2) caches where you want entries to be evicted under memory pressure; (3) listener registries where listeners can be GC'd without explicit deregistration. SoftReference: GC'd only when memory is low -- better for caches.

Q15. What is the invokedynamic bytecode instruction?

invokedynamic (Java 7, JSR 292) enables dynamic method dispatch at runtime. On first call, a bootstrap method is invoked to create a CallSite with a MethodHandle target. Subsequent calls dispatch directly via the MethodHandle -- very efficient. Used for: (1) **Lambdas** -- LambdaMetafactory bootstrap creates functional interface instances; (2) **String concatenation** (Java 9+) -- StringConcatFactory replaces StringBuilder chains; (3) Dynamic languages on the JVM (Groovy, Kotlin, Scala).

Q16. What is the difference between soft, weak, and phantom references?

Soft: cleared by GC when memory is low (JVM tries to hold on as long as possible). `get()` returns null after clearing. Used for memory-sensitive caches. **Weak:** cleared at any GC, even if memory is plentiful. `get()` returns null after. Used for canonicalization maps, metadata. **Phantom:** always returns null from `get()`. Enqueued in `ReferenceQueue` after object is finalized. Used for pre-mortem cleanup without using `finalize()`. Safer than finalizers.

Q17. How do you detect a memory leak in a Java application?

Signs: heap usage grows over time, increasing Full GC frequency, eventual OOM. Approach: (1) Enable GC logging, analyse heap usage trend with `jstat` or GC log tools; (2) Take heap dump: `-XX:+HeapDumpOnOutOfMemoryError` or `jcmd GC.heap_dump`; (3) Analyse with Eclipse MAT: dominator tree, leak suspects, retained heap; (4) Look for: large static collections, growing `ThreadLocal` maps, undrained queues, unclosed streams, unsealed listener lists.

Q18. What is JFR and why is it preferred over traditional profilers?

Java Flight Recorder is a JVM-embedded continuous profiler. Advantages: (1) **Low overhead** -- typically less than 1% (safepoint-less sampling for most events); (2) **Always-on** -- can be enabled in production continuously; (3) **Rich data** -- GC, thread, I/O, lock, allocation, CPU, network in one recording; (4) **Circular buffer** -- last N minutes always available on crash/OOM; (5) Custom events via JFR API. Traditional profilers (JProfiler, YourKit) have 2-5% overhead.

Q19. What is the purpose of the Constant Pool in a class file?

The Constant Pool is a table of symbolic references and literal values referenced by the class. It stores: class names, field/method names and descriptors, string literals, int/float/long/double constants, and `MethodHandle/MethodType` entries (Java 7+). All symbolic references in bytecodes (`invokevirtual`, `getfield`, etc.) index into the constant pool. During resolution, symbolic references are replaced with direct pointers. The runtime constant pool (in Metaspace) is the JVM's version of the class file constant pool.

Q20. What is TLAB (Thread-Local Allocation Buffer)?

TLAB is a private region of Eden allocated to each thread for object allocation. Allocation within a TLAB uses a simple bump pointer -- extremely fast (as fast as C `malloc`). No synchronization needed since only one thread writes to its TLAB. When a TLAB is exhausted, the thread requests a new one (which requires a brief lock). Large objects that exceed TLAB size are allocated directly in Eden or Old Gen. TLABs dramatically reduce allocation contention in multi-threaded applications.

Q21. What flags would you use to diagnose high GC pause times?

Essential flags: `-Xlog:gc*:file=gc.log:time,uptime` (unified logging, Java 9+). Analysis: look for pause duration, GC cause (allocation failure, metadata GC threshold), young/old gen sizes. Tools: GCViewer, GCEasy.io for log analysis. Additional: `-XX:+PrintGCDetails`, `-XX:+PrintGCDateStamps` (Java 8). For lock profiling: `-Xlog:safepoint`. For allocation profiling: JFR `jdk.ObjectAllocationInNewTLAB` event.

Q22. What is object promotion in GC?

Object promotion is the process of moving a surviving object from the Young Generation to the Old Generation. Conditions for promotion: (1) Object's age (number of Minor GCs survived) exceeds `MaxTenuringThreshold` (default 15); (2) Object is too large for Survivor space; (3) Survivor space is full (overflow promotion). Premature promotion: objects promoted too early fill Old Gen, triggering expensive Major GC. Fix: increase Survivor space size or `MaxTenuringThreshold`.

Q23. What is the difference between major version and minor version in a class file?

The class file has a major and minor version number encoding the required JVM version. Major version 52 = Java 8, 55 = Java 11, 61 = Java 17, 65 = Java 21. A JVM only loads classes with major version `<=` its own version. Loading a class compiled for Java 17 on a Java 11 JVM throws: `UnsupportedClassVersionError: MyClass has been compiled by a more recent version. Minor version 65535 (0xFFFF) indicates a class uses Java preview features.`

Q24. What is a safepoint in the JVM?

A safepoint is a point in execution where the JVM can safely perform operations that require all application threads to be paused (stop-the-world). The JVM signals threads to reach the nearest safepoint -- threads check a flag at: method return, loop back-edges, uncounted loops. Operations at safepoints: GC, deoptimisation, stack trace dumps, class redefinition. Issue: a thread in a long-counted loop may delay reaching a safepoint -- 'safepoint bias'. Monitor with `-Xlog:safepoint`.

Q25. What is deoptimisation in the JIT compiler?

Deoptimisation transfers execution from JIT-compiled native code back to the interpreter. Triggers: (1) Speculative assumption violated -- e.g., devirtualised call sees a new subtype; (2) Class loading makes a previously made-final type non-final; (3) Exception happens in a compiled method (uncommon trap). The JIT compiles 'uncommon trap' stubs for rare conditions. On deopt: frame is reconstructed from compiled state into interpreter format. Monitor with `-XX:+PrintDeoptimization`.

Q26. What is object header in HotSpot and how much memory does an object use?

Every Java object has a 12-16 byte header (HotSpot): (1) **Mark word** (8 bytes): GC age, hash code, lock state (biased lock removed Java 21); (2) **Klass pointer** (4 bytes compressed / 8 bytes uncompressed): pointer to class metadata. Array objects have an additional 4-byte length field. Minimum object size: 16 bytes (with compressed oops, default). int field: 4 bytes. Object reference: 4 bytes (compressed oops, `-XX:+UseCompressedOops`, default for heap < 32GB). JOL (Java Object Layout) library measures actual object sizes.

Interview Q&A; - Advanced Questions

[ADV] Q27. How does G1 GC's concurrent marking work and what is a concurrent mode failure?

G1 concurrent marking runs in background while the application runs: (1) Initial Mark (STW, piggybacked on Young GC): marks roots; (2) Root Region Scan (concurrent): scan Survivor regions for pointers to Old Gen; (3) Concurrent Mark (concurrent): trace object graph from roots, build liveness data; (4) Remark (STW): finalise marking using SATB (Snapshot-At-The-Beginning) write barrier; (5) Cleanup (STW + concurrent): identify empty regions, account for liveness. Concurrent mode failure: application allocates faster than marking can keep up -- G1 falls back to a full STW collection. Fix: trigger marking earlier (lower `InitiatingHeapOccupancyPercent`).

[ADV] Q28. Explain ZGC's colored pointer technique.

ZGC uses 64-bit pointers with metadata bits (colored pointers): bits 42-45 encode: Marked0, Marked1, Remapped, Finalizable. Every pointer read goes through a **load barrier** -- native code inserted by the JIT checks pointer color and performs relocation/remapping if needed. This allows ZGC to relocate objects concurrently while the application runs -- any read of an old pointer self-heals by following the forwarding pointer. Trade-off: every object read has a small load barrier cost (~3-5 ns). Benefit: all GC phases except 3 brief STW pauses are concurrent.

[ADV] Q29. What is the SATB (Snapshot-At-The-Beginning) write barrier in G1?

SATB is a write barrier technique for concurrent marking. When a reference is overwritten (object A no longer points to object B), the **previous value** (B) is recorded in a thread-local SATB buffer. The concurrent marker processes these buffers to ensure B (and its descendants) are marked even though A no longer references it. This prevents the tri-color invariant violation: a black object (all refs scanned) referring to an unmarked white object -- which would cause live objects to be collected. Remark STW phase processes remaining SATB buffers.

[ADV] Q30. How does the JVM handle class unloading?

A class can be unloaded only when its `ClassLoader` is garbage collected. The `ClassLoader` is GC'd only when: no live references to the `ClassLoader` itself, no live references to any Class object loaded by it, and no live references to instances of those classes. Bootstrap CL classes are never unloaded. App CL classes rarely unload. Dynamic class loaders (e.g., in OSGi, app servers) unload classes when contexts are destroyed. Enable: `-XX:+TraceClassUnloading`. Failure to unload: `ClassLoader` leak -- Metaspace grows.

[ADV] Q31. What is on-stack replacement (OSR) in the JIT compiler?

On-Stack Replacement (OSR) allows the JVM to switch from interpreted to JIT-compiled code while a method is executing (mid-loop). Without OSR, a long-running loop would run entirely in the interpreter until the method returns. Trigger: back-edge counter exceeds threshold in a loop. The JVM compiles the method and transfers execution to the compiled version at the loop header. OSR compiled code is suboptimal (must account for interpreter state). After the method exits and re-enters, it uses the standard compiled version.

[ADV] Q32. Explain the difference between HotSpot's C1 and C2 JIT compilers.

C1 (client compiler): fast compilation, moderate optimisations. Produces profiled code that collects type, branch, and invocation data. Low compilation overhead -- fast startup. C2 (server compiler): slow, aggressive compilation using profile data. Applies: aggressive inlining, loop unrolling, auto-vectorisation (SIMD), partial escape analysis, speculative devirtualisation, global code motion. C2 compilation can take 100ms+ but produces highly optimised native code. Tiered compilation: C1 for early execution, C2 for hot methods after profiling. GraalVM: replaces C2 with Graal compiler (Java-written, pluggable).

[ADV] Q33. What is the Humongous region in G1 GC and what problems can it cause?

Humongous regions hold objects larger than 50% of a G1 region size. They are allocated in contiguous Old Gen regions and never moved. Problems: (1) **Fragmentation** -- contiguous regions may not be available; (2) **Premature Full GC** -- humongous allocation can fail if no contiguous regions; (3) **Eager reclamation** -- G1 reclaims humongous objects during concurrent marking (when only weakly reachable), which can interfere with regular GC pauses. Fix: increase region size (`-XX:G1HeapRegionSize`) to reduce what qualifies as humongous.

[ADV] Q34. How does the JVM implement invokedynamic for lambda expressions?

Lambda compilation (`LambdaMetafactory`): the compiler emits an `invokedynamic` instruction with a bootstrap method call to `LambdaMetafactory.metafactory()`. At first invocation: bootstrap generates (via ASM) a class implementing the functional interface, with an instance method calling the target lambda body (a private static method in the enclosing class). A `CallSite` is returned pointing to the factory method. Subsequent calls: `CallSite` dispatch is as fast as a direct `MethodHandle` invocation. No anonymous class generated at compile time (unlike Java 7 anonymous classes).

[ADV] Q35. What is compressed ordinary object pointers (compressed oops) and when does it not work?

On 64-bit JVMs, object references would be 8 bytes. `CompressedOops` (`-XX:+UseCompressedOops`, default) stores 4-byte offsets (shifted left by 3 bits = 8-byte alignment) into the heap base address. Supports heaps up to 32 GB (4 bytes * 8 = 32 GB addressable). Does NOT work when: heap > 32 GB (`-Xmx > 32g`) -- references become 8 bytes. Transition from 28-32 GB heap can INCREASE memory usage (reference sizes double). Also: `CompressedClassPointers` compresses klass pointers separately (4 bytes instead of 8).

[ADV] Q36. Explain the JVM's memory model for object layout and padding.

HotSpot lays out objects as: header (mark word + klass pointer), then fields in order: longs/doubles, ints/floats, chars/shorts, bytes/booleans, references (to minimise padding). Fields are aligned to their size (int at 4-byte boundary, long at 8-byte boundary). The JVM may insert padding between fields for alignment. Object size is rounded up to 8 bytes. Field reordering is controlled by the JVM, not declaration order. Use JOL (Java Object Layout) to inspect actual layout: `ClassLayout.parseClass(Foo.class).toPrintable()`.

[ADV] Q37. What is Epsilon GC and what is it used for?

Epsilon GC (Java 11, `-XX:+UseEpsilonGC`) is a no-op garbage collector -- it allocates memory but never performs any GC. The JVM terminates with OOM when heap is full. Use cases: (1) **Performance testing** -- measure allocation rate without GC interference; (2) **Ultra-short-lived applications** -- if the app exits before heap fills (serverless nano-functions); (3) **GC overhead measurement** -- compare Epsilon vs other GCs to quantify GC cost; (4) **GraalVM native image** -- AOT apps may not need GC at all.

[ADV] Q38. How does the JVM perform safepoint-biased sampling and what is the problem?

Traditional JVM profilers (like JVisualVM) sample thread stacks at safepoints. Safepoints only occur at specific locations (method returns, loop back-edges). Problem: code between safepoints (e.g., inside a tight counted loop) is **never sampled** -- profiler shows exit points, not where time is actually spent. This is safepoint bias -- profiling data is misleading for CPU-intensive code. Solution: **async-profiler** uses OS signals (SIGPROF) to sample at any arbitrary point regardless of safepoints -- accurate CPU profiling.

[ADV] Q39. What is card table and how does it support generational GC?

The card table is a compact bitmap (1 byte per 512-byte heap region = 'card'). When a reference in Old Gen is updated to point to a Young Gen object, the JVM's **write barrier** marks the corresponding card as 'dirty'. During Minor GC: instead of scanning all Old Gen objects, only dirty cards are scanned for cross-generational references (Old-to-Young pointers = GC roots for Young Gen). This reduces Minor GC work from $O(\text{heap})$ to $O(\text{dirty cards})$, making it fast.

[ADV] Q40. Explain how string interning works in the JVM.

`String.intern()` returns the canonical instance of a string from the string pool. In Java 6-: string pool was in PermGen (fixed size, risk of OOM). Java 7+: string pool moved to heap (GC'd normally). Compile-time string literals are automatically interned. String pool uses a hashtable structure. `intern()` returns existing entry or adds a new one. Size: `-XX:StringTableSize=65536` (default buckets). Useful for memory reduction when many duplicate strings exist (e.g., status codes, country codes). Not appropriate for general strings -- `intern()` has overhead.

[ADV] Q41. What is the JVM's approach to Just-In-Time compilation of virtual calls?

Virtual calls (`invokevirtual`, `invokeinterface`) cannot be inlined without knowing the receiver type. C1/C2 collect type profiles: (1) **Monomorphic** (1 type seen) -- inline the target method with a null/type guard; (2) **Bimorphic** (2 types) -- if/else inline both; (3) **Megamorphic** (3+ types) -- cannot inline, use vtable dispatch. If a new type appears after monomorphic inlining: deoptimise, recompile as bimorphic. Framework code with many subclasses (Spring beans) often megamorphic -- inlining impossible.

[ADV] Q42. How does the JVM's write barrier work in G1 GC?

G1 uses two write barriers: (1) **Pre-write barrier (SATB)**: before a reference field is overwritten, the old value is enqueued in an SATB buffer for the concurrent marker. (2) **Post-write barrier**: after a reference field is written, if the write crosses a region boundary (old region -> young region), the Remembered Set (RSet) of the target young region is updated. RSets allow G1 to find all pointers into a region without scanning the whole heap -- essential for collecting individual regions efficiently.

[ADV] Q43. What is the difference between reachability and liveness for GC purposes?

Reachability: an object is reachable if it can be reached by traversing any strong reference chain from GC roots (thread stacks, static fields, JNI refs). Only strongly reachable objects are guaranteed to survive GC.

Liveness (logical): an object is live if it will actually be used in the future. The JVM can only determine reachability, not logical liveness. An object may be strongly reachable but logically dead (held by a static collection but never used again). This is the essence of a Java memory leak -- GC cannot collect logically dead but reachable objects.

[ADV] Q44. Explain how JVM implements string concatenation with invokedynamic (Java 9+).

Java 9+ uses `StringConcatFactory.makeConcatWithConstants()` as the bootstrap method for invokedynamic-based string concatenation. At first invocation, the bootstrap method generates (via `MethodHandles`) an efficient concatenation strategy -- typically direct `byte[]` construction without intermediate `StringBuilder`. Strategies: `BC_SB` (`StringBuilder`-based), `MH_SB_SIZED` (pre-sized), `MH_INLINE_SIZED_EXACT` (most efficient). Benefit: JIT can see the concatenation as a single operation and optimise it; no bytecode overhead of multiple `StringBuilder` calls.

[ADV] Q45. What is Graal compiler and how does it relate to HotSpot's C2?

The Graal compiler is a high-performance JIT compiler written in Java (replacing C2). It is part of GraalVM and can be used as a drop-in replacement: `-XX:+UseJVMCICompiler`. Advantages over C2: (1) Written in Java -- easier to maintain and extend; (2) More aggressive partial escape analysis; (3) Better inlining decisions (Graal IR vs C2 sea-of-nodes); (4) Foundation for AOT compilation (GraalVM Native Image). GraalVM Community Edition is open-source; Enterprise edition has additional optimisations. Spring Native, Quarkus, Micronaut use GraalVM for native image compilation.

[ADV] Q46. How does the JVM handle out-of-memory conditions gracefully?

When heap is exhausted: JVM runs full GC; if still insufficient, throws `OutOfMemoryError`. Handling strategies: (1) `-XX:+HeapDumpOnOutOfMemoryError` -- auto heap dump for analysis; (2) `-XX:+ExitOnOutOfMemoryError` -- exit immediately (container-friendly, let orchestrator restart); (3) `-XX:+CrashOnOutOfMemoryError` -- crash dump for deep analysis; (4) Custom thread `UncaughtExceptionHandler` to log and gracefully degrade; (5) `OOMKiller` in Linux containers -- configure `-Xmx` below container memory limit. Different OOM types: heap space, Metaspace, Direct buffer memory, unable to create native thread.

[ADV] Q47. What is the Remembered Set (RSet) in G1 GC and what is its overhead?

A Remembered Set (RSet) is maintained per region -- it records which regions contain pointers into that region. This allows G1 to collect any region as a GC root set without scanning the whole heap. Implementation: per-region sparse/coarse/fine-grained card sets. RSet overhead: typically 1-5% of heap. For Humongous objects: no RSet (too large; scanned from all regions instead). Post-write barrier updates RSets asynchronously via refinement threads. High write rates: dirty card queues grow, refinement threads fall behind -- causes GC pauses.

[ADV] Q48. Explain the JVM's approach to lock optimisations: thin locks, fat locks, and lock coarsening.

Thin lock (lightweight lock): uses CAS to store a displaced mark word in the stack frame. Fast path for uncontended locks -- no OS involvement. Fat lock (inflated): on contention, the thin lock inflates to a monitor (mutex). OS thread blocking involved. Lock coarsening: JIT merges adjacent synchronized blocks into one to reduce lock/unlock overhead. Lock elimination: JIT removes locks on non-escaping objects (escape analysis). Adaptive spinning: before blocking, the JVM spins briefly hoping the lock is released (`-XX:+UseSpinning`, `-XX:PreBlockSpin`).

[ADV] Q49. How does JFR's low overhead recording work internally?

JFR uses a lock-free, thread-local buffer design: each thread has a local buffer (~1KB). Events are written to the thread-local buffer without any lock -- very low overhead. Buffers are periodically flushed to a global ring buffer (disk if duration-limited). Stack trace sampling uses async signal (SIGPROF-like) or safepoint samples. For object allocation events: JFR instruments TLAB allocation paths -- only fires when TLAB fills (amortised over many allocations). Disk I/O events: instrumented at JVM level, not OS level. Total overhead typically 1-3% even with all events enabled.

[ADV] Q50. What is the class file verification process and why can it be skipped?

Bytecode verification ensures class files are structurally correct and type-safe before execution. Checks: valid opcodes, correct operand stack depths, type safety of operations, valid local variable accesses, superclass is not final, no infinite loops in initialiser, etc. Phases: format check (loading), pass 1 (loading), pass 2 (preparation), pass 3-4 (linking). Skipping: -Xverify:none -- disables verification for performance (Java 13+: deprecated). Safe to skip: if all classes come from trusted, pre-verified sources (internal app code). Unsafe to skip: any untrusted code (user-uploaded classes).

[ADV] Q51. Explain how CMS (Concurrent Mark Sweep) differs from G1 and why it was removed.

CMS was the first concurrent GC in HotSpot. Old Gen collection is mostly concurrent (no stop-the-world except initial mark and remark). Problems: (1) CMS is a non-compacting collector -- leads to fragmentation over time; (2) Fragmentation causes promotion failures triggering costly Full GC with compaction; (3) Concurrent mode failure: if allocation rate exceeds collection rate, Full GC falls back; (4) Complex tuning with many flags. G1 solves fragmentation via region-based compaction. CMS deprecated Java 9, removed Java 14.

[ADV] Q52. What is the relationship between JVM bytecode and the Java class file magic number 0xCAFEBABE?

Every Java .class file begins with the 4-byte magic number 0xCAFEBABE (hexadecimal). It was chosen by James Gosling (Java's creator) as a whimsical identifier. The JVM checks for this magic number before attempting to load a class -- if absent, throws `ClassFormatError`. It provides a fast sanity check that a file is a valid Java class file. Following CAFEBABE: minor version (2 bytes), major version (2 bytes). Example: Java 17 class = CA FE BA BE 00 00 00 3D (0x3D = 61 decimal).

Coding Exercises

Exercise 1: Custom ClassLoader for Hot-Reload

```
public class HotReloadClassLoader extends ClassLoader {
    private final Path classDir;

    public HotReloadClassLoader(Path classDir, ClassLoader parent) {
        super(parent);
        this.classDir = classDir;
    }

    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        Path classFile = classDir.resolve(name.replace('.', '/') + ".class");
        if (Files.exists(classFile)) {
            try {
                byte[] bytes = Files.readAllBytes(classFile);
                return defineClass(name, bytes, 0, bytes.length);
            } catch (IOException e) {
                throw new ClassNotFoundException(name, e);
            }
        }
        return super.findClass(name);
    }

    // To hot-reload: create a NEW instance of this classloader
    // (old classes are unloaded when old classloader is GC'd)
    public static Object reload(String className, Path classDir) throws Exception {
        HotReloadClassLoader cl = new HotReloadClassLoader(classDir,
            Thread.currentThread().getContextClassLoader());
        Class<?> clazz = cl.loadClass(className);
        return clazz.getDeclaredConstructor().newInstance();
    }
}
```

Exercise 2: Heap & GC Monitoring via MXBeans

```
public class JvmMonitor {
    public static void printMemoryStats() {
        MemoryMXBean mem = ManagementFactory.getMemoryMXBean();
        System.out.printf("Heap used: %d MB%n",
            mem.getHeapMemoryUsage().getUsed() / 1_048_576);
        System.out.printf("Heap committed: %d MB%n",
            mem.getHeapMemoryUsage().getCommitted() / 1_048_576);
        System.out.printf("Heap max: %d MB%n",
            mem.getHeapMemoryUsage().getMax() / 1_048_576);
        System.out.printf("Non-heap used: %d MB%n",
            mem.getNonHeapMemoryUsage().getUsed() / 1_048_576);
    }

    public static void printGCStats() {
        for (GarbageCollectorMXBean gc : ManagementFactory.getGarbageCollectorMXBeans()) {
            System.out.printf("GC: %-20s Collections: %d Time: %d ms%n",
                gc.getName(), gc.getCollectionCount(), gc.getCollectionTime());
        }
    }

    public static void printThreadStats() {
        ThreadMXBean threads = ManagementFactory.getThreadMXBean();
        System.out.printf("Thread count: %d (peak: %d, daemon: %d)%n",
            threads.getThreadCount(),
            threads.getPeakThreadCount(),
```

```

threads.getDaemonThreadCount());
}
}

```

Exercise 3: Weak Reference Cache

```

public class WeakRefCache<K, V> {
    private final ConcurrentHashMap<K, WeakReference<V>> cache = new ConcurrentHashMap<>();
    private final Function<K, V> loader;

    public WeakRefCache(Function<K, V> loader) { this.loader = loader; }

    public V get(K key) {
        WeakReference<V> ref = cache.get(key);
        V value = ref != null ? ref.get() : null;
        if (value == null) {
            value = loader.apply(key);
            cache.put(key, new WeakReference<>(value));
        }
        return value;
    }

    // Periodically evict stale entries (GC'd values)
    public void evictStale() {
        cache.entrySet().removeIf(e -> e.getValue().get() == null);
    }
}

```

Exercise 4: JFR Custom Event

```

import jdk.jfr.*;

@Name("com.example.OrderProcessed")
@Label("Order Processed")
@Category({"Business", "Orders"})
@Description("Fired when an order is processed")
@StackTrace(false)
public class OrderProcessedEvent extends Event {
    @Label("Order ID") public String orderId;
    @Label("Amount") public double amount;
    @Label("Status") public String status;
}

// Usage in service
public void processOrder(Order order) {
    OrderProcessedEvent event = new OrderProcessedEvent();
    event.begin();
    try {
        // ... actual processing ...
        event.orderId = order.getId();
        event.amount = order.getAmount();
        event.status = "SUCCESS";
    } catch (Exception e) {
        event.status = "FAILED";
        throw e;
    } finally {
        event.commit(); // only records if JFR recording is active
    }
}

```

Debug & Fix Scenarios

Debug 1: ClassLoader Memory Leak (Metaspace Growth)

```
// BUG: new classloader created per request, old ones never GC'd
// Metaspace grows until OOM: Metaspace
public class DynamicProcessor {
    public Object process(byte[] classBytes, String className) throws Exception {
        URLClassLoader cl = new URLClassLoader(urls); // BUG: new CL each call
        Class<?> clazz = cl.loadClass(className); // classes accumulate in Metaspace
        return clazz.getDeclaredConstructor().newInstance();
        // cl is never closed -- stays reachable via classBytes -> class -> classloader
    }
}

// Fix: close the classloader when done, or cache and reuse it
public Object processFixed(byte[] classBytes, String className) throws Exception {
    try (URLClassLoader cl = new URLClassLoader(urls)) { // auto-close
        Class<?> clazz = cl.loadClass(className);
        return clazz.getDeclaredConstructor().newInstance();
    } // cl.close() allows eventual class unloading
}
```

Debug 2: String Interning OOM (Java 6 PermGen)

```
// BUG: interning many unique strings exhausts PermGen / StringTable
List<String> ids = new ArrayList<>();
for (long i = 0; i < 1_000_000; i++) {
    ids.add("ID-" + i).intern()); // BUG: 1M unique interned strings
// In Java 6: fills PermGen string pool -> OOM
// In Java 7+: heap strings, but string table chaining degrades performance
}

// Fix: don't intern unique strings; only intern high-cardinality repeated values
// Use a bounded cache if deduplication is needed:
private static final Map<String, String> INTERN_CACHE =
    Collections.synchronizedMap(new LinkedHashMap<>(1000, 0.75f, true) {
        protected boolean removeEldestEntry(Map.Entry e) { return size() > 1000; }
    });
public static String safeIntern(String s) {
    return INTERN_CACHE.computeIfAbsent(s, Function.identity());
}
```

Debug 3: Premature Promotion Causing Frequent Full GC

```
// Symptom: frequent Major/Full GC, short-lived objects in Old Gen
// Diagnosis: jstat -gcutil <pid> 1000 -- 0% keeps growing
// GC log: many 'Promotion failed' or 'to-space exhausted'
// Cause: Survivor spaces too small, objects promoted before they die
// -XX:SurvivorRatio=8 means Eden=8, S0=1, S1=1 (of Young Gen)
// If many objects survive past S0/S1 capacity, they're promoted early
// Fix options:
// 1. Increase Young Generation size: -Xmn1g (or -XX:G1NewSizePercent=30)
// 2. Decrease SurvivorRatio to increase Survivor spaces: -XX:SurvivorRatio=4
// 3. Lower MaxTenuringThreshold to let objects die in Survivor longer is wrong
// INCREASE it: -XX:MaxTenuringThreshold=15 (default; objects need more cycles)
// 4. For G1: -XX:G1NewSizePercent=20 -XX:G1MaxNewSizePercent=40
// 5. Profile allocation hotspots with JFR: jdk.ObjectAllocationInNewTLAB event
```

Debug 4: ClassNotFoundException due to Wrong ClassLoader Context

```
// BUG: ClassNotFoundException in multi-classloader environment (app server)
```

```
// Thread's context classloader is different from the one that loaded the class
// Common in: JNDI lookups, JDBC drivers, XML parsers in Java EE containers
Thread.currentThread().getContextClassLoader()
.loadClass("com.example.SomeService"); // may fail in container
// Fix: temporarily set context classloader
ClassLoader original = Thread.currentThread().getContextClassLoader();
try {
    Thread.currentThread().setContextClassLoader(
        SomeService.class.getClassLoader()); // use the right classloader
    // ... perform JNDI/JDBC lookup ...
} finally {
    Thread.currentThread().setContextClassLoader(original); // ALWAYS restore
}
```

Multiple Choice Questions

Q1. Which JVM memory area is shared between all threads?

- A) JVM Stack
- B) PC Register
- C) Heap [Correct]
- D) Native Method Stack

Q2. What replaced PermGen in Java 8?

- A) Code Cache
- B) Old Generation
- C) Metaspace [Correct]
- D) Direct Memory

Q3. What is the first 4 bytes of every valid Java .class file?

- A) 0xDEADBEEF
- B) 0xCAFEBAFE [Correct]
- C) 0xFEEFFACE
- D) 0xBAADF00D

Q4. Which GC algorithm is the default in Java 9+?

- A) Parallel GC
- B) CMS GC
- C) Serial GC
- D) G1 GC [Correct]

Q5. What JIT compilation level uses C2 (optimising compiler)?

- A) Level 1
- B) Level 2
- C) Level 3
- D) Level 4 [Correct]

Q6. Which reference type is cleared by GC only when memory is low?

- A) Weak Reference
- B) Strong Reference
- C) Soft Reference [Correct]
- D) Phantom Reference

Quick Reference Cheat Sheet

Concept	Key Detail	Flag / Tool
Heap	Shared; objects + arrays; GC-managed	-Xms/-Xmx
Metaspace	Class metadata in native memory; no hard cap by default	-XX:MaxMetaspaceSize=256m
JVM Stack	Per-thread; frames with locals + operand stack	-Xss512k
TLAB	Per-thread Eden bump-pointer allocation; no sync needed	-XX:TLABSize
Minor GC	Young Gen only (Eden full); STW, fast; copies survivors	-XX:SurvivorRatio, -Xmn
Full GC	Entire heap; STW, slow; triggered by OOM risk or System.gc()	-XX:+DisableExplicitGC
G1 GC	Region-based; concurrent marking; mixed GC; default Java 9+	-XX:MaxGCPauseMillis=200
ZGC	Colored pointers; load barriers; sub-ms pauses; Java 15+ prod	-XX:+UseZGC
Tiered JIT	Level 0 (interp) -> C1 (profiling) -> C2 (optimised)	-XX:+TieredCompilation
Method Inlining	Most impactful JIT opt; enables escape analysis	-XX:MaxInlineSize=35
Escape Analysis	Stack-alloc / lock elision for non-escaping objects	-XX:+DoEscapeAnalysis
invokedynamic	Dynamic dispatch; used for lambdas + String concat	javap -v to inspect
Compressed Oops	4-byte refs for heaps < 32 GB; default on 64-bit JVMs	-XX:+UseCompressedOops
WeakReference	Cleared at any GC; for caches and canonical maps	WeakHashMap
SoftReference	Cleared only when memory low; memory-sensitive caches	SoftReference(obj)
Safepoint	All threads paused for GC, deopt, stack scan	-Xlog:safepoint
Card Table	Tracks Old->Young references; makes Minor GC fast	Write barrier (JVM internal)
JFR	Low-overhead production profiler (<1%); built into JDK	-XX:StartFlightRecording